

Ansible lab report breakdown

This lab demonstrates end-to-end network automation using Ansible on Windows Subsystem for Linux (WSL Ubuntu 24.04) connected to a Cisco Catalyst 9000 IOS XE device (v17.15.1) on the Cisco DevNet Always-On Sandbox. The lab covered environment setup, inventory management, playbook development, Jinja2 templating, dry-run deployment, resource modules, and automated reporting.

Parts Completed

Part	Topic	What Was Done
A & B	Environment Setup	Installed Ubuntu WSL2, Python venv, Ansible Core 2.21, cisco.ios and ansible.netcommon collections, built project folder structure
C	Inventory & Variables	Defined device in YAML inventory; separated connection params into group_vars and device metadata into host_vars
D	Connectivity Testing	Verified DNS, SSH handshake, and Ansible ad hoc command execution against the live Catalyst 9000 sandbox
E	Playbooks	4 playbooks: show commands, structured fact gathering (saved as JSON), OS version compliance check, and config backup
F	Jinja2 Templating	Two Jinja2 templates from YAML data: loopback interface config and NTP/syslog/banner management services config
G	Dry-Run Deployment	Used --check --diff mode; Ansible confirmed 4 CLI commands would be sent to create Loopback150 without applying any changes
H	Resource Modules	ios_interfaces in rendered state (YAML to CLI offline) and gathered state (live interface state returned as structured JSON)
I	Markdown Reporting	Generated formatted device report from the same YAML variables used for configuration — single source of truth

Challenges & Fixes

- community.general YAML callback removed in v12.0.0 — fixed by switching to result_format=yaml in ansible.cfg
- ansible.netcommon.network_cli connection string caused errors — fixed by using network_cli shorthand with a vars block in each playbook
- DevNet Always-On sandbox credentials rotated mid-lab — resolved by switching to the Catalyst 9000 sandbox with verified credentials
- Jinja2 template delegation to localhost lost group_vars context — fixed by declaring variables explicitly in the playbook vars block

Key Lessons Learned

- Ansible cleanly separates inventory, variables, templates, and playbooks — each has a distinct, testable role
- Network devices require the network_cli connection plugin, not standard Linux SSH modules
- Check mode and diff mode are essential safety tools — always dry-run before pushing config to a shared device
- Structured YAML data can drive both configuration generation and documentation from a single source of truth